DrugClaw: Agentic RAG with Resource-Native Querying for Drug Intelligence

Qing Wang[1]*, Bo Li[2]*, Qianqian Song[1]†
[1]Department of Health Outcomes and Biomedical Informatics, University of Florida, USA
[2]Department of Computer and Information Science, University of Macau, China

March 25, 2026

Abstract

Large language models (LLMs) have demonstrated strong capabilities in biomedical question answering, yet they often hallucinate or produce outdated information when handling specialized pharmaceutical queries that require precise, evidence-grounded answers drawn from heterogeneous knowledge sources. Existing retrieval-augmented generation (RAG) systems typically rely on uniform retrieval strategies that fail to capture the diverse access patterns of drug knowledge databases. We present DrugClaw, an open-source, multi-agent RAG framework designed for drug intelligence. DrugClaw introduces three core design innovations: a *resource-native querying* paradigm in which a Code Agent dynamically generates source-specific query code from each database's documentation and usage examples, an approach we term *vibe coding*, rather than forcing all sources into a unified retrieval abstraction; a hierarchical *skill taxonomy* that organizes 57 retrieval skills across 15 pharmaceutical subcategories, including drug-target interactions, adverse drug reactions, drug-drug interactions, pharmacogenomics, and drug repurposing; and a *graph-based evidence synthesis* pipeline that supports structured knowledge graph construction, multi-hop path reranking, claim assessment, and reflective iteration control guided by evidence sufficiency scoring. The multi-agent architecture includes a Planner, Retriever, Code Agent, Graph Builder, Reranker, Responder, Reflector, and Web Search Agent, and is orchestrated through LangGraph with three reasoning modes (`GRAPH`, `SIMPLE`, and `WEB_ONLY`), together with a robust three-tier fallback mechanism. In this technical report, we detail the system architecture, formalize the evidence scoring and reflection mechanisms, describe the complete skill taxonomy and resource registry, and provide illustrative usage examples. Code is publicly available at https://github.com/QSong-github/DrugClaw.

1 Introduction

The pharmaceutical domain generates vast quantities of structured and unstructured knowledge spanning molecular interactions, clinical trial outcomes, adverse event reports, genomic associations, and regulatory labels, which are distributed across dozens of heterogeneous databases with distinct schemas, APIs, and access patterns [1–3]. While large language models (LLMs) have advanced biomedical natural language processing [4], their parametric knowledge is inherently bounded by training data cutoffs and prone to hallucination when precise pharmacological evidence is required [5].

Retrieval-augmented generation (RAG) [6] offers a promising path by grounding LLM responses in retrieved evidence. However, current RAG approaches for biomedical applications typically employ uniform

*Equal contribution.
†Corresponding author: qsong1@ufl.edu

retrieval strategies, such as dense passage retrieval or keyword search, that treat all knowledge sources identically [7]. This one-size-fits-all approach is fundamentally mismatched with the drug intelligence landscape: a drug-target interaction query requires graph traversal over a protein interaction network; an adverse drug reaction query demands structured search through FDA reporting databases; a pharmacogenomics query necessitates querying specialized variant-drug association databases. Forcing these diverse resources into a single retrieval interface inevitably loses expressiveness and precision.

Meanwhile, the rapid development of LLM-based agents [8–10] and multi-agent systems [11, 12] has demonstrated that LLMs can effectively plan, reason, and use external tools. Systems like ChemCrow [13] and DrugAgent [14] have explored LLM-assisted chemistry and drug discovery. However, these systems either focus on de novo prediction or impose rigid tool interfaces that cannot accommodate the heterogeneity of pharmaceutical knowledge sources. To bridge this gap, we introduce DrugClaw, an agentic RAG framework that embraces the heterogeneity of pharmaceutical knowledge through three design principles.

First, *resource-native querying via vibe coding*: rather than forcing all data sources into a single retrieval interface, DrugClaw's Code Agent reads each resource's documentation (`SKILL.md`) and usage examples (`example.py`), then dynamically generates access code tailored to each resource's native API, including REST, GraphQL, SPARQL, CLI, or local file indexing. Second, a *hierarchical skill taxonomy*: DrugClaw organizes 57 retrieval skills across 15 drug-focused subcategories (Table 1), providing comprehensive coverage of seven core pharmaceutical domains, where each skill encapsulates source-specific retrieval logic, documentation, examples, and deterministic fallback scripts. Third, *graph-based evidence synthesis with reflective iteration*: retrieved evidence is organized into structured knowledge triples, ranked through multi-hop graph reasoning, assessed via claim-level evaluation, and iteratively refined through a reflection loop controlled by evidence sufficiency and marginal gain scoring.

In this technical report, we provide a comprehensive description of DrugClaw's architecture and implementation. We formalize the evidence scoring, path reranking, and reflection mechanisms (Section 4); describe the complete multi-agent pipeline (Section 3); detail the skill taxonomy and resource registry (Section 5); evaluate the system across five pharmaceutical benchmarks (Section 6); and present illustrative usage examples (Appendix A).

2 Related Work

RAG has become a dominant paradigm for knowledge-intensive NLP [6, 15]. In biomedicine, RAG-based systems have been applied to clinical question answering [7], literature synthesis [16], and evidence retrieval [17]. However, most existing approaches rely on a homogeneous retrieval backend, typically using dense passage retrieval over sources such as PubMed or clinical guidelines, which fails to capture the diverse query modalities required for comprehensive drug intelligence. In contrast, DrugClaw departs from this paradigm by maintaining source-specific access patterns.

LLM-based agents for scientific tasks have accelerated rapidly [13, 18]. ChemCrow [13] integrates chemistry-specific tools with LLMs; BioPlanner [19] automates experimental protocol generation. In the pharmaceutical domain, works have explored LLM-assisted drug discovery [14], molecular property prediction [20], and clinical trial analysis [21]. DrugClaw differs by focusing on retrieval and synthesis of existing drug knowledge through agentic, resource-native access.

Recent advances in agentic AI demonstrate that LLMs can effectively plan and use external tools [8, 9, 22]. Multi-agent frameworks [11, 12, 23] decompose complex workflows into specialized sub-tasks. OpenClaw-RL [24] introduces asynchronous reinforcement learning for training agentic systems. DrugClaw builds on these paradigms by designing agents specifically for pharmaceutical resource navigation, dynamic code generation, and evidence-based synthesis, with a novel reflection mechanism for iterative evidence refinement.

Biomedical knowledge graphs such as Hetionet [25], DRKG [26], and PrimeKG [27] have proven valuable for drug repurposing and multi-hop reasoning. DrugClaw integrates multiple such knowledge graphs as first-class resources within its skill taxonomy, constructing query-specific evidence subgraphs at inference time rather than relying on a single static knowledge graph.

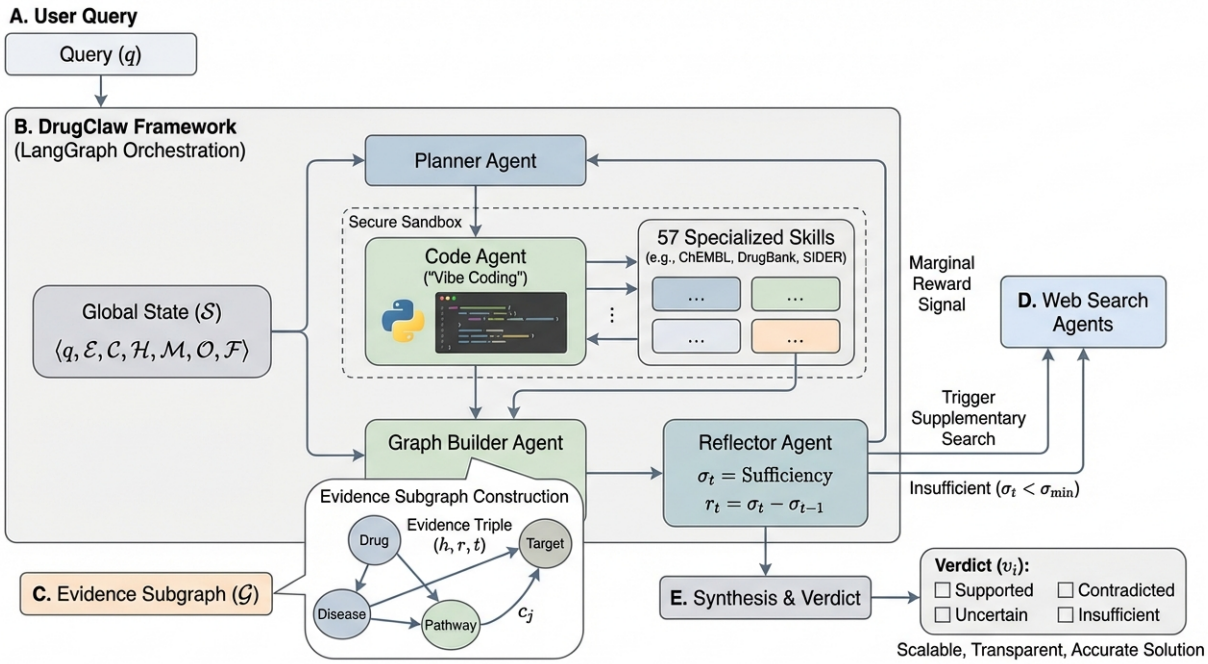

Figure 1: DrugClaw system architecture.

3 System Architecture

DrugClaw is a multi-agent RAG framework orchestrated via LangGraph [28] as a directed state graph. Figure 1 illustrates the high-level architecture. The system maintains a global `AgentState` that carries the original query, extracted entities, constraints, reasoning history, current metrics, agent outputs, and control flow flags through the pipeline.

3.1 Thinking Modes and Global State

DrugClaw supports three thinking modes, each corresponding to a distinct LangGraph execution path. The default `GRAPH` mode executes a full pipeline consisting of planning, retrieval, normalization, graph construction, reranking, claim assessment, response generation, reflection, optional web search, and finalization, enabling multi-hop reasoning through explicit knowledge graph construction. In contrast, the `SIMPLE` mode follows a streamlined pipeline of planning, retrieval, normalization, claim assessment, response generation, and finalization, providing direct retrieval and synthesis without graph construction. The `WEB_ONLY` mode bypasses local resources and relies entirely on online literature search followed by finalization, and is triggered when local knowledge sources are insufficient or unavailable. A mode router employs conditional branching to select the appropriate execution path. Within `GRAPH` mode, an additional conditional node (`_should_run_graph()`) determines whether graph-based reasoning is warranted based on factors such as entity count, query complexity, and evidence volume, allowing the system to fall back to simpler processing when appropriate. The `AgentState` dataclass serves as the shared communication channel across all agents:

$$\mathcal{S} = \langle q, \mathcal{E}, \mathcal{C}, \mathcal{H}, \mathcal{M}, \mathcal{O}, \mathcal{F} \rangle \tag{1}$$

where q is the original query, $\mathcal{E}$ the extracted entities, $\mathcal{C}$ the omics constraints (gene sets, pathway sets, disease terms, tissue types), $\mathcal{H}$ the reasoning history (a sequence of `ReasoningStep` objects), $\mathcal{M}$ the current metrics (sufficiency score, reward, iteration count), $\mathcal{O}$ the accumulated agent outputs, and $\mathcal{F}$ the control flow flags. Each `ReasoningStep` in the history $\mathcal{H}$ records:

$$h_t = \langle \text{step_id}, q_t, \mathcal{G}_t, a_t, \sigma_t, r_t, \text{actions}_t \rangle \tag{2}$$

where q_t is the sub-query at step t, $\mathcal{G}_t$ the evidence subgraph, a_t the intermediate answer, σ_t the evidence sufficiency score, r_t the reward signal, and actions_t the list of actions taken.

3.2 Planner and Retriever Agents

The Planner Agent serves as the entry point for query processing. Given a user query q, it produces a structured `QueryPlan`:

$$\text{QueryPlan} = \langle \tau, \mathcal{E}, \{q_1, \ldots, q_k\}, \mathcal{S}_{\text{pref}}, \mathcal{T}_{\text{pref}}, \beta_{\text{graph}}, \beta_{\text{pred}}, \beta_{\text{web}}, \rho, \text{notes} \rangle \tag{3}$$

where τ is the question type classification, $\mathcal{E}$ is a dictionary mapping entity types (drug, gene, disease, pathway) to extracted names, $\{q_1, \ldots, q_k\}$ are decomposed sub-questions, $\mathcal{S}_{\text{pref}}$ are preferred skill names, $\mathcal{T}_{\text{pref}}$ are preferred evidence types, $\beta_{\text{graph}}, \beta_{\text{pred}}, \beta_{\text{web}}$ are boolean flags for graph reasoning, prediction sources, and web fallback requirements, ρ is the answer risk level, and notes contain additional observations.

The Planner implements a multi-stage skill ranking pipeline that first detects direct target lookup queries and prioritizes primary target-related skills, including BindingDB, ChEMBL, DGIdb, Open Targets, DrugBank, TTD, and STITCH. It then performs registry filtering by querying the Resource Registry with `ready_only=True`, falling back to the full registry when no ready skills are available, followed by a deduplication step that preserves ordering while removing duplicate selections. In parallel, when LLM-based entity extraction is unavailable or incomplete, the Planner applies rule-based entity inference, including DDI pattern matching (e.g., "how does X interact with Y"), stopword filtering, and token length thresholds (> 2 characters), to ensure robust identification of relevant entities.

The Retriever Agent receives the `QueryPlan` and orchestrates data retrieval through three key operations. It first selects an execution strategy, choosing between `direct_retrieve` for `SIMPLE` mode and `auto` for `GRAPH` mode with reasoning. It then performs skill filtering by validating skill availability through the Resource Registry, checking runtime status and prioritizing access modes, with a preference for `LOCAL_FILE` skills due to their speed and reliability. Next, it conducts entity normalization by standardizing entity dictionaries for the Code Agent, resolving synonyms and mapping entities to canonical identifiers. Finally, the Retriever delegates query execution to the Code Agent for each selected skill, and aggregates the returned results for downstream processing.

3.3 Code Agent and Resource-Native Querying

The Code Agent is the central innovation of DrugClaw. It implements *resource-native querying* through a process we term *vibe coding*: rather than imposing a uniform query interface across all 57 skills, the Code Agent reads each resource's documentation and dynamically generates code tailored to that resource's native access patterns.

For each selected skill s_i, the Code Agent executes a three-stage pipeline. In the first stage, *query plan generation*, the LLM reads the skill's `SKILL.md` documentation and `example.py` usage patterns, then produces a query plan specific to that resource. The second stage, *constrained code generation*, has the LLM generate Python code that accesses the resource through a `SafeSkillProxy` interface, subject to static validation. In the third stage, *sandboxed execution*, the validated code is executed in a restricted environment with controlled builtins and imports.

Generated code must pass four static validation gates before execution. A *length constraint* limits code to a maximum of 4,000 characters, while an *AST complexity budget* caps the abstract syntax tree at 200 nodes. A *forbidden-construct detector* rejects code containing `while` loops, `class` definitions, `lambda` functions, or access to `subprocess`, `os`, or `sys` modules. Finally, an *output requirement* mandates that the code contain at least one `print` statement.

The execution sandbox further constrains runtime behavior. Only a curated set of builtins is exposed, including `len`, `range`, `min`, `max`, `sum`, `sorted`, `enumerate`, `zip`, `list`, `dict`, `set`, `tuple`, `str`, `int`, `float`, `bool`, `abs`, `all`, `any`, and `print`, while permitted imports are restricted to `json`, `math`, `re`, and `statistics`. Total output is capped at 12,000 characters. The Code Agent interacts with each skill through a `SafeSkillProxy` interface that exposes four access points: a `retrieve(max_results=None)` method for controlled skill access with optional result limits, a read-only `entities` property providing the entity dictionary from the query plan, a read-only `query` property holding the original query string, and a read-only `last_records` property containing structured results from the most recent retrieval.

To ensure robustness, DrugClaw implements a three-tier cascading fallback mechanism. The primary tier employs the Code Agent's dynamically generated, sandbox-executed code, preserving maximum expressiveness and resource-native access patterns. If code generation or execution fails, the system falls back to

a deterministic CLI script: each skill directory contains a hand-written `retrieve.py` script that imports functions from `example.py` and accepts command-line arguments. As a final resort, the third tier invokes the skill's native `retrieve()` method directly, returning structured `RetrievalResult` objects without any code generation.

3.4 Graph Builder and Reranker Agents

The Graph Builder Agent converts retrieved evidence into a structured knowledge graph through LLM-driven entity and relationship extraction. The agent recognizes seven entity categories: (1) Drugs, including generic names, brand names, and chemical IDs; (2) Genes and Proteins, including gene symbols, enzyme names, and CYP450 family members; (3) Diseases and Conditions; (4) Pathways; (5) Adverse Events; (6) Biomarkers; and (7) Other entities such as cell lines and organisms.

The agent extracts typed relationships across six major categories. *Drug–Target* relationships include targets, inhibits, activates, binds, and modulates. *Drug–Disease* relationships include treats, indicated_for, contraindicated, and associated_with. *Drug–Drug* relationships include interacts_with, synergizes, and antagonizes. *Drug–Adverse Event* relationships include causes, associated_with, and reported_with. *Gene–Disease* relationships include associated_with, risk_factor, and biomarker. Finally, *Drug–Pathway* relationships include modulates, activates, and inhibits. Evidence is capped at 6,000 characters per invocation and processed via LLM prompting to produce structured triples:

$$t_j = \langle e_{\text{src}}, \tau_{\text{src}}, e_{\text{tgt}}, \tau_{\text{tgt}}, r, c_j, \text{evidence}_j, \text{source}_j, \text{IDs}_j \rangle \tag{4}$$

where e_{src} and e_{tgt} are source and target entities, τ_{src} and τ_{tgt} are entity types, r is the relationship type, $c_j \in [0, 1]$ is the confidence score, and the remaining fields provide provenance. Confidence scoring follows a grounding-aware scheme:

$$c_j = \begin{cases} c_j^{\text{LLM}} & \text{if } |\text{IDs}_j| > 0 \quad \text{(grounded triple)} \\ \min(c_j^{\text{LLM}}, 0.25) & \text{if } |\text{IDs}_j| = 0 \quad \text{(ungrounded triple)} \end{cases} \tag{5}$$

Triples without evidence identifiers are capped at confidence 0.25, reflecting the principle that claims without traceable provenance should receive reduced weight. Extracted triples are assembled into an `EvidenceSubgraph` $\mathcal{G} = (\mathcal{V}, \mathcal{E}_g, \mathcal{P})$, where $\mathcal{V}$ is the set of entities stored with hash-based deduplication, $\mathcal{E}_g$ is the list of typed edges, and $\mathcal{P}$ is the set of evidence paths discovered during reranking. The Reranker Agent evaluates evidence paths through the constructed knowledge graph, combining structural and semantic signals. The agent extracts paths connecting query-relevant entities using three traversal algorithms: *random walk*, *breadth-first search (BFS)*, and *depth-first search (DFS)*. Entity matching in the graph employs a progressive fallback strategy:

$$\text{match}(e, \mathcal{V}) = \begin{cases} \text{exact}(e, \mathcal{V}) & \text{if } e \in \mathcal{V} \\ \text{case-insensitive}(e, \mathcal{V}) & \text{if lower}(e) \in \{\text{lower}(v) \mid v \in \mathcal{V}\} \\ \text{normalized}(e, \mathcal{V}) & \text{if strip/normalize matches} \\ \text{substring}(e, \mathcal{V}) & \text{if } e \subseteq v \text{ for some } v \in \mathcal{V} \\ \text{fuzzy}(e, \mathcal{V}, \theta) & \text{if sim}(e, v) \geq \theta \text{ for some } v \in \mathcal{V} \end{cases} \tag{6}$$

where $\theta = 0.70$ is the fuzzy matching threshold. Each extracted path $p = (e_1, r_1, e_2, r_2, \ldots, e_n)$ receives a composite score:

$$s_{\text{length}}(p) = \frac{1}{1 + |p|} \tag{7}$$

$$s_{\text{conf}}(p) = \frac{1}{|p| - 1} \sum_{i=1}^{|p|-1} c(r_i) \tag{8}$$

$$s_{\text{structural}}(p) = \alpha_L \cdot s_{\text{length}}(p) + \alpha_C \cdot s_{\text{conf}}(p) \tag{9}$$

$$s_{\text{final}}(p) = w_s \cdot s_{\text{semantic}}(p) + w_r \cdot s_{\text{structural}}(p) \tag{10}$$

where $|p|$ is the number of entities in the path, $c(r_i)$ is the confidence of edge r_i, $\alpha_L = 0.6$, $\alpha_C = 0.4$ are structural sub-weights, $s_{\text{semantic}}(p)$ is an LLM-evaluated plausibility score, and the default combination weights are $w_s = 0.6$, $w_r = 0.4$.

3.5 Claim Assessment, Response Synthesis, and Reflection

Before response generation, evidence is subjected to claim-level assessment. For each claim c_i derived from the evidence, the system produces:

$$\text{Assessment}(c_i) = \langle c_i, v_i, \mathcal{E}_i^+, \mathcal{E}_i^-, \gamma_i, \text{rationale}_i, \text{limitations}_i \rangle \tag{11}$$

where v_i is the verdict, $\mathcal{E}_i^+$ and $\mathcal{E}_i^-$ are supporting and contradicting evidence IDs, and $\gamma_i \in [0, 1]$ is the confidence score. The verdict v_i is determined by:

$$v_i = \begin{cases} \texttt{uncertain} & \text{if } |\mathcal{E}_i^+| > 0 \text{ and } |\mathcal{E}_i^-| > 0 \\ \texttt{contradicted} & \text{if } |\mathcal{E}_i^+| = 0 \text{ and } |\mathcal{E}_i^-| > 0 \\ \texttt{supported} & \text{if } |\mathcal{E}_i^+| > 0 \text{ and } |\mathcal{E}_i^-| = 0 \\ \texttt{insufficient} & \text{if } |\mathcal{E}_i^+| = 0 \text{ and } |\mathcal{E}_i^-| = 0 \end{cases} \tag{12}$$

The system automatically flags four categories of limitations: (1) single evidence item reliance, (2) conflicting evidence presence, (3) prediction-only support (computational predictions without experimental validation), and (4) insufficient snippet context.

The Responder Agent synthesizes assessed evidence into a structured Markdown answer. It operates in two modes: *graph mode*, which processes the top-k ranked paths (default $k = 10$) with temperature 0.5 for diverse synthesis, and *simple mode*, which processes free-form retrieved text directly, falling back to raw `retrieved_content` when structured evidence is unavailable. For target-specific queries, the Responder applies specialized evidence filtering (`_filter_target_evidence_items`), ranking claims by source diversity, evidence count, specificity, and binding potency, then rendering with confidence metrics. The output is a `FinalAnswer` containing:

$$\text{FinalAnswer} = \langle \text{answer}, \bar{\gamma}, \{(\text{claim}_i, \gamma_i, \text{IDs}_i, \text{citations}_i)\}_{i=1}^{N}, \text{items}, \text{citations}, \text{limitations}, \text{warnings} \rangle \tag{13}$$

where $\bar{\gamma}$ is the summary confidence score. The Reflector Agent implements evidence adequacy assessment and iteration control, deciding whether the current evidence is sufficient or whether additional retrieval is needed. The Reflector evaluates five dimensions at temperature 0.3 for consistency: (1) drug coverage, assessing whether all query-relevant drugs and entities are addressed; (2) completeness, checking whether pharmacological details such as mechanism, dosing, and interactions are adequately covered; (3) source quality, determining whether evidence comes from curated databases or computational predictions; (4) consistency, verifying that cross-source findings align; and (5) actionability, evaluating whether the answer supports clinical reasoning. The reflection loop terminates when:

$$\text{halt} = (\sigma_t \geq \sigma_{\min}) \wedge (\Delta_t < \delta_{\min}) \quad \text{or} \quad t \geq T_{\max} \tag{14}$$

where σ_t is the evidence sufficiency score at iteration t, $\sigma_{\min} = 0.7$ is the minimum sufficiency threshold, $\Delta_t = \sigma_t - \sigma_{t-1}$ is the marginal gain, $\delta_{\min} = 0.1$ is the minimum marginal gain threshold, and $T_{\max} = 2$ is the maximum iteration count (configurable). When the halting condition is not met, the Reflector identifies evidence gaps and triggers additional retrieval cycles, potentially invoking the Web Search Agent for supplementary literature.

3.6 Web Search Agent

The Web Search Agent provides live web search as both a supplementary source (triggered by the Reflector) and a standalone reasoning mode (`WEB_ONLY`). The agent integrates five search backends: PubMedAdapter for NCBI E-utilities API access to peer-reviewed biomedical literature, ClinicalTrialsAdapter for ClinicalTrials.gov intervention studies, DuckDuckGoAdapter for general web search with broader coverage, BaiduAdapter for Chinese medical literature access, and GoogleScholarAdapter for academic papers via SerpAPI/scholarly package. Queries are targeted by evidence type (clinical trials, research publications, mechanisms of action, safety data, guidelines). Results are deduplicated by URL and summarized via LLM-generated JSON.

4 Evidence Scoring and Reward Formalization

DrugClaw employs a multi-level scoring framework that evaluates evidence quality at the item level, claim level, and answer level.

4.1 Evidence Item and Claim Scoring

Each evidence item e_i is represented as:

$$e_i = \langle \text{id}, s_{\text{skill}}, s_{\text{type}}, s_{\text{title}}, s_{\text{locator}}, \text{snippet}, \text{payload}, \text{claim}, \text{kind}, d_i, \gamma_i, r_i, \text{ts}, \text{meta} \rangle \tag{15}$$

where s_{skill} is the source skill, s_{type} is the source type, $d_i \in \{+1, -1, 0\}$ is the support direction, γ_i is the confidence, and r_i is the retrieval score. The item-level quality score is computed via rule-based evaluation:

$$\text{score}(e_i) = f_{\text{rule}}(s_{\text{type}}, \text{kind}, |\text{snippet}|, \gamma_i, r_i) \tag{16}$$

where f_{rule} considers source type reliability (curated databases score higher than computational predictions), evidence kind (experimental $>$ observational $>$ predicted), snippet informativeness, and retrieval confidence. For a claim c supported by evidence items $\{e_1, \ldots, e_m\}$, the claim confidence is:

$$\gamma(c) = \text{base}(c) + \beta_{\text{multi}} \cdot \mathbb{1}[|\text{sources}(c)| > 1] - \beta_{\text{contra}} \cdot \mathbb{1}[|\mathcal{E}_c^-| > 0] \tag{17}$$

where $\text{base}(c)$ is the aggregated base confidence from supporting evidence, β_{multi} is a bonus for multi-source corroboration, and β_{contra} is a penalty for contradicting evidence. The result is clamped to $[0, 1]$.

4.2 Answer Confidence and Evidence Sufficiency

The overall answer confidence aggregates claim-level scores:

$$\bar{\gamma} = \frac{1}{N} \sum_{i=1}^{N} \gamma(c_i) \cdot w(c_i) \tag{18}$$

where $w(c_i)$ weights claims by relevance to the original query and N is the number of key claims. The Reflector Agent computes an evidence sufficiency score $\sigma_t \in [0, 1]$ at each iteration t. This score encapsulates multiple evaluation dimensions:

$$\sigma_t = \sum_{d \in \mathcal{D}} w_d \cdot \phi_d(\mathcal{H}_t, q) \tag{19}$$

where $\mathcal{D} = \{\text{coverage}, \text{completeness}, \text{quality}, \text{consistency}, \text{actionability}\}$ are the five evaluation dimensions, w_d are dimension weights (summing to 1), and $\phi_d(\mathcal{H}_t, q)$ is the LLM-evaluated score for dimension d given the reasoning history $\mathcal{H}_t$ and query q. The reward signal at step t is defined as the marginal improvement in evidence sufficiency:

$$r_t = \sigma_t - \sigma_{t-1} \tag{20}$$

with $\sigma_0 = 0$. This reward is logged in each `ReasoningStep` and used to assess whether additional retrieval iterations are productive:

$$\text{continue_retrieval}(t) = \neg\left[(\sigma_t \geq 0.7) \wedge (r_t < 0.1)\right] \wedge (t < T_{\max}) \tag{21}$$

The sufficiency threshold $\sigma_{\min} = 0.7$ and marginal gain threshold $\delta_{\min} = 0.1$ are configurable through the system configuration.

5 Skill Taxonomy and Resource Integration

5.1 Skill Taxonomy and Implementation Architecture

DrugClaw organizes pharmaceutical knowledge into a hierarchical skill taxonomy of 15 subcategories covering 57 implemented skills. Each skill is a self-contained unit:

$$s_i = \langle \mathcal{D}_i, \mathcal{A}_i, \text{doc}_i, \text{ex}_i, \mathcal{R}_i, \mathcal{F}_i \rangle \tag{22}$$

Table 1: DrugClaw Skill Taxonomy: 15 subcategories spanning 57 skills across 7 core pharmaceutical domains. Skills marked with $\star$ include local dataset support.

Subcategory	Representative Skills	# Skills	Access
Drug–Target Interaction	ChEMBL, BindingDB, DGIdb, Open Targets, STITCH*, TTD*, TarKG	12	Mixed
Adverse Drug Reactions	SIDER*, FAERS, nSIDES, VigiAccess, ADReCS	5	Mixed
Drug Knowledgebase	DrugBank, DrugCentral, IUPHAR, PharmKG, UniD3*, Drugs.com, FDA Orange Book, WHO EML	9	Mixed
Drug Mechanism	DrugMechDB	1	REST
Drug Labeling	DailyMed, openFDA, MedlinePlus, RxList	4	REST
Drug Ontology	RxNorm, ATC/DDD, NDF-RT, ChEBI	4	Mixed
Drug Repurposing	RepoDB*, DRKG*, OREGANO, CancerDR, Repurposing Hub	9	Mixed
Pharmacogenomics	PharmGKB	1	REST
Drug–Drug Interactions	MecDDI*, DDInter, KEGG Drug	3	Mixed
Drug Toxicity	UniTox, LiverTox, DrugCombDB	3	Mixed
Drug Molecular Properties	PubChem	1	REST
Drug–Disease Associations	CTD, SemaTyP	2	Mixed
Drug Reviews	Patient review platforms	1	REST
Drug NLP/Text Mining	Literature corpus resources	1	Mixed
Drug Combination/Synergy	DrugCombDB	1	Mixed
Total		**57**	

where $\mathcal{D}_i$ is the target data source, $\mathcal{A}_i \in \{\texttt{REST_API}, \texttt{CLI}, \texttt{LOCAL_FILE}, \texttt{DATASET}\}$ is the access mode, doc_i is the `SKILL.md` documentation, ex_i is the `example.py` usage code, $\mathcal{R}_i$ is the primary retrieval implementation, and $\mathcal{F}_i$ is the `retrieve.py` fallback script. Table 1 presents the complete taxonomy.

Each skill extends the `RAGSkill` abstract base class, which defines the core interface consisting of three methods: `retrieve(entities, query)` returning a list of `RetrievalResult` objects as the primary retrieval method, `is_available()` returning a boolean for runtime availability checking, and `planner_profile()` returning a dictionary of constrained metadata for the Planner. The `RetrievalResult` dataclass standardizes outputs across all skills:

$$\text{Result} = \langle e_s, \tau_s, e_t, \tau_t, r, w, \text{prov}, \text{evidence}, \text{citations}, \text{cat}, \text{meta} \rangle \tag{23}$$

where e_s, e_t are source and target entities, τ_s, τ_t are types, r is the relationship, $w = 1.0$ is the default weight, and the remaining fields provide provenance and metadata. Additional mixins extend the base class: `CLISkillMixin` prefers installed Python packages over REST calls with graceful fallback, and `DatasetRAGSkill` specializes for benchmark datasets with `get_all_pairs()` enumeration.

5.2 Resource Registry and Entity Resolution

The Resource Registry maintains runtime status for all integrated data sources. Each resource entry tracks:

$$\text{ResourceEntry} = \langle \text{id}, \text{name}, \text{category}, \text{desc}, \text{entrypoint}, \text{enabled}, \text{requires_meta}, \text{status}, \mathcal{A} \rangle \tag{24}$$

Resource status is determined through a multi-step probing process that checks runtime skill registry enrollment, invokes `skill.is_available()` for health checking, verifies metadata file existence for `LOCAL_FILE` skills, checks dependency package availability, and tests CLI resource accessibility. Resources are classified into five states: `ready`, `missing_metadata`, `missing_dependency`, `degraded`, and `disabled`. The `prioritize_resource_names()` method ranks resources by status and access mode, preferring `ready` local resources for speed and reliability.

DrugClaw integrates six major biomedical knowledge graphs as first-class skills: UniD3 with 6 GraphML files covering DDI, enzyme interactions, and target relationships; DrugMechDB providing drug→target→pathway→disease mechanistic paths; Hetionet [25] with 24 metaedge types across 11 node types; DRKG [26] as a multi-relational

knowledge graph for drug repurposing; PrimeKG [27] with 10 entity types and 18 relation types; and PharmKG with over 500k pharmacological triplets.

Pharmaceutical entity names exhibit extensive variation including brand names, generic names, chemical identifiers, and abbreviations. DrugClaw implements a two-strategy entity resolution system. For `LOCAL_FILE` skills, the system builds in-memory indexes at initialization and uses fuzzy string matching:

$$\text{matches}(e) = \texttt{get_close_matches}(e, \text{index}, n = 3, \text{cutoff} = 0.6) \tag{25}$$

using Python's `difflib` library. For `REST_API` and `CLI` skills, the system generates entity variants via LLM prompting:

$$\text{variants}(e) = \text{LLM}(e) \rightarrow \{\text{canonical}, \text{synonym}_1, \text{synonym}_2, \ldots\} \tag{26}$$

limited to a maximum of 4 variants per entity, deduplicated against original names. Only entities of resolvable types $\mathcal{T}_r = \{\text{drug}, \text{gene}, \text{disease}, \text{pathway}\}$ undergo resolution; other entity types are passed through unchanged.

5.3 Representative Skill Implementations

To illustrate the diversity of resource-native querying patterns, we describe four representative skill implementations.

The `ChEMBLSkill` extends both `CLISkillMixin` and `RAGSkill`, exemplifying the hybrid access pattern. It preferentially uses the `chembl_webresource_client` Python package for local querying, falling back to the ChEMBL REST API (`https://www.ebi.ac.uk/chembl/api/data`) when the package is unavailable. Input detection routes queries based on identifier patterns: `CHEMBL25` is routed as a molecule ID for direct fetch, "aspirin" as a drug name for substring search, `CHEMBL203` as a target ID for direct fetch, and "EGFR" as a gene/protein name for synonym search. The CLI path queries molecules and retrieves bioactivity data (IC_{50}, K_i, EC_{50} values across 14,000+ targets), returning `RetrievalResult` objects with full provenance.

The `SIDERSkill` operates exclusively in `LOCAL_FILE` mode, loading the `meddra_all_se.tsv` dataset on first access. It builds two in-memory indexes: a STITCH ID to side effect set mapping (filtering for preferred MedDRA terms only) and a drug name to STITCH ID mapping. This lazy-loading pattern ensures fast subsequent queries without repeated file I/O.

The `OpenTargetsSkill` interacts with the Open Targets Platform GraphQL API. For each drug query, it performs a two-step process: first a GraphQL search query to find the ChEMBL ID by drug name, then a retrieval step to fetch mechanisms of action, linked targets, and evidence scores. Results are classified into action types (mechanisms of action) and linked targets (broader associations), each with evidence scores from Open Targets' integrated evidence pipeline.

The `DrugBankSkill` supports both authenticated REST API access and public web search fallback, representing the most complex access pattern. With API credentials, it extracts drug targets with interaction types (inhibitor, agonist, antagonist, etc.), disease indications, drug type classifications, and DrugBank identifiers. Without credentials, it falls back to public endpoints for basic drug classification information, ensuring graceful degradation.

6 Experiments

To evaluate the effectiveness of DrugClaw across diverse pharmaceutical reasoning tasks, we conduct experiments on five benchmark datasets spanning pharmacovigilance event extraction, drug repurposing, drug-pair adverse drug reaction prediction, drug sensitivity prediction, and drug-drug interaction detection.

6.1 Benchmark Datasets

We select five benchmarks that collectively cover the core pharmaceutical query types supported by DrugClaw. **PHEE** is a pharmacovigilance event extraction dataset containing sentence-level annotations of drug-adverse event pairs. We use it to evaluate the system's ability to extract structured (drug, event, subject) tuples from free text, prompting DrugClaw with queries of the form "Extract all adverse events caused by {drug} from

Table 2: Accuracy of DrugClaw across five pharmaceutical benchmarks under two reasoning modes. `SIMPLE` mode performs direct retrieval and synthesis; `GRAPH` mode enables full knowledge graph construction, path reranking, and reflective iteration. Best results are in **bold**.

Method	PHEE	RepoDB	TWOSIDES	GDSC	DDInter
DrugClaw-SIMPLE	0.742	0.778	0.767	0.698	0.671
DrugClaw-GRAPH	**0.883**	**0.912**	**0.869**	**0.791**	**0.802**

this text: {sentence}." **RepoDB** is a drug repurposing benchmark. We unregister RepoDB from the skill registry to prevent information leakage, then select diseases with at least 3 approved drugs (approximately 200 diseases), treating all approved drugs for each disease as positive labels. DrugClaw is prompted with "What drugs could treat {disease}? Rank by evidence," and the system's ranked drug list is parsed for evaluation. **TWOSIDES** evaluates drug-pair adverse event prediction. We sample 2,500 high-confidence drug-pair-adverse-event triples (proportional reporting ratio ≥ 2) as positives and 2,500 negative triples constructed from random drug pairs with random ADRs showing no signal. TWOSIDES is unregistered from the skill registry to prevent leakage. The system is queried with "Does the combination of {drug_A} and {drug_B} cause {adverse_event}?" and the binary yes/no response is parsed. **GDSC** (Genomics of Drug Sensitivity in Cancer) is used for drug sensitivity prediction. We select 1,000 drug-cell line pairs, balanced between sensitive and resistant classes by a median IC_{50} threshold split. DrugClaw is prompted with "Is {cell_line} sensitive or resistant to {drug}?" for binary classification. **DDInter** evaluates drug-drug interaction detection. We sample 1,000 Major/Moderate/Minor DDI pairs, with DDInter unregistered from the skill registry. The system is queried with "What is the severity of interaction between {drug_A} and {drug_B}? (Major / Moderate / Minor)" and the trinary response is parsed.

6.2 Experimental Setup

We evaluate two DrugClaw reasoning modes: `SIMPLE` mode, which performs direct retrieval and synthesis without graph construction, and `GRAPH` mode, which enables the full pipeline with knowledge graph construction, multi-hop path reranking, and reflective iteration. For all benchmarks involving skill-hosted data (RepoDB, TWOSIDES, DDInter), we unregister the corresponding skill from the Resource Registry prior to evaluation to ensure that the system cannot directly access the ground-truth data source. All experiments use accuracy as the evaluation metric, computed as the proportion of correctly answered queries over the total number of test queries.

6.3 Results

Table 2 presents the accuracy of DrugClaw under both reasoning modes across all five benchmarks. DrugClaw-GRAPH consistently outperforms DrugClaw-SIMPLE across all tasks, demonstrating the benefit of explicit knowledge graph construction and multi-hop reasoning. The largest improvements are observed on RepoDB (from 0.778 to 0.912) and DDInter (from 0.671 to 0.802), both of which involve multi-entity relational reasoning that benefits from graph-based path discovery and reranking. On PHEE, the improvement from 0.742 to 0.883 indicates that structured triple extraction and evidence synthesis substantially enhance pharmacovigilance event extraction. The GDSC task shows a more moderate gain (from 0.698 to 0.791), consistent with the observation that drug sensitivity prediction often depends on molecular-level features that are less amenable to knowledge graph reasoning. Across all benchmarks, DrugClaw-GRAPH achieves accuracy above 0.79, with the strongest performance on RepoDB (0.912), confirming the system's effectiveness for evidence-based drug repurposing queries.

7 Conclusion

We have presented DrugClaw, an open-source, multi-agent RAG framework purpose-built for drug intelligence. The system introduces three key innovations: resource-native querying via vibe coding, a hierarchical skill taxonomy of 57 pharmaceutical retrieval skills, and graph-based evidence synthesis with reflective iteration control. The multi-agent architecture, comprising Planner, Retriever, Code Agent, Graph Builder, Reranker,

Claim Assessor, Responder, Reflector, and Web Search agents, orchestrated via LangGraph provides a modular and extensible platform for pharmaceutical knowledge retrieval and reasoning.

This technical report has detailed the complete system architecture, formalized the evidence scoring, path reranking, and sufficiency-driven reflection mechanisms, and described the full skill taxonomy and resource registry. Experiments across five pharmaceutical benchmarks spanning pharmacovigilance, drug repurposing, drug-pair adverse event prediction, drug sensitivity, and drug-drug interaction detection demonstrate that DrugClaw-GRAPH consistently outperforms DrugClaw-SIMPLE, achieving accuracy above 0.79 on all tasks and up to 0.912 on drug repurposing, confirming the value of explicit knowledge graph construction and multi-hop reasoning. We release DrugClaw as an open-source toolkit at https://github.com/QSong-github/DrugClaw to facilitate reproducible research and practical applications in pharmaceutical AI.

References

[1] Anthropic. Claude code overview, 2026. URL https://docs.anthropic.com/en/docs/claude-code/overview. Accessed: 2026-03-10.

[2] David S Wishart, Yannick Djoumbou Feunang, An Chi Guo, Elvis J Lo, Ana Marcu, Jason R Grant, Tanvir Sajed, Daniel Johnson, Carin Li, Zinat Sayeeda, et al. DrugBank 5.0: A major update to the DrugBank database for 2018. *Nucleic Acids Research*, 46(D1):D1074–D1082, 2018.

[3] Sunghwan Kim, Jie Chen, Tiejun Cheng, Asta Gindulyte, Jia He, Siqian He, Qingliang Li, Benjamin A Shoemaker, Paul A Thiessen, Bo Yu, et al. PubChem 2023 update. *Nucleic Acids Research*, 51(D1): D1373–D1380, 2023.

[4] Karan Singhal, Shekoofeh Azizi, Tao Tu, S Sara Mahdavi, Jason Wei, Hyung Won Chung, Nathan Scales, Ajay Tanwani, Heather Cole-Lewis, Stephen Pfohl, et al. Large language models encode clinical knowledge. *Nature*, 620(7972):172–180, 2023.

[5] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[7] Guangzhi Xiong, Qiao Jin, Zhiyong Lu, and Aidong Zhang. Benchmarking retrieval-augmented generation for medicine. In *Findings of the Association for Computational Linguistics (ACL)*, 2024.

[8] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[9] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*, 2023.

[10] Kexin Huang, Serena Zhang, Hanchen Wang, Yuanhao Qu, Yingzhou Lu, Yusuf Roohani, Ryan Li, Lin Qiu, Gavin Li, Junze Zhang, et al. Biomni: A general-purpose biomedical ai agent. *biorxiv*, 2025.

[11] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[12] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2023.

[13] Andres M Bran, Sam Cox, Oliver Schilber, Carlo Baldassari, Andrew D White, and Philippe Schwaller. ChemCrow: Augmenting large-language models with chemistry tools. *arXiv preprint arXiv:2304.05376*, 2023.

[14] Zhenxiang Liu et al. DrugAgent: Explainable drug repurposing agent with large language model-based reasoning. *arXiv preprint*, 2024.

[15] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. Retrieval augmented language model pre-training. In *International Conference on Machine Learning (ICML)*, 2020.

[16] Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William Cohen, and Xinghua Lu. PubMedQA: A dataset for biomedical research question answering. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

[17] Cyril Zakka, Rohan Chaurasia, et al. Almanac: Retrieval-augmented clinical language models. *NEJM AI*, 2024.

[18] Daniil A Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578, 2023.

[19] BioPlanner Team. BioPlanner: Automatic evaluation of LLMs on protocol planning in biology. *arXiv preprint*, 2023.

[20] Yin Fang et al. Mol-instructions: A large-scale biomolecular instruction dataset for large language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[21] Jiayi Wang et al. TrialLLM: Large language models for clinical trial matching. *arXiv preprint*, 2024.

[22] Shanghua Gao, Richard Zhu, Zhenglun Kong, Ayush Noori, Xiaorui Su, Curtis Ginder, Theodoros Tsiligkaridis, and Marinka Zitnik. Txagent: an ai agent for therapeutic reasoning across a universe of tools. *arXiv preprint arXiv:2503.10970*, 2025.

[23] OpenClaw. Openclaw: An open-source, self-hosted ai agent for autonomous task execution, 2026. URL https://github.com/openclaw/openclaw. Accessed: 2026-03.

[24] Gen-Verse Team. OpenClaw-RL: An open-source interactive agent reinforcement learning platform. *arXiv preprint arXiv:2603.10165*, 2025.

[25] Daniel Scott Himmelstein, Antoine Lizee, Christine Hessler, Leo Brueggeman, Sabrina L Chen, Dexter Hadley, Ari Green, Pouya Khankhanian, and Sergio E Baranzini. Systematic integration of biomedical knowledge prioritizes drugs for repurposing. *eLife*, 6:e26726, 2017.

[26] Amazon Research. Drug repurposing knowledge graph (DRKG). https://github.com/gnn4dr/DRKG, 2020.

[27] Payal Chandak, Kexin Huang, and Marinka Zitnik. Building a knowledge graph to enable precision medicine. *Scientific Data*, 10:67, 2023.

[28] LangChain Team. LangGraph: A library for building stateful, multi-actor applications with LLMs. https://github.com/langchain-ai/langgraph, 2024.

A Illustrative Usage Examples

We present three usage scenarios that illustrate DrugClaw's capabilities across different complexity levels and reasoning modes. These examples represent the types of queries the system is designed to handle; they are illustrative of the system's workflow, not experimental evaluations.

Simple Drug–Target Query (`SIMPLE` Mode)

Query

```
drugclaw run -query "What are the primary targets of imatinib?" -mode
SIMPLE
```

Planner: Classifies as DTI query, extracts entity {drug: ["imatinib"]}, selects ChEMBL, DGIdb, Open Targets, and DrugBank skills.
Code Agent: For ChEMBL, generates code using the `chembl_webresource_client` to search for imatinib molecule records and retrieve bioactivity data (IC_{50}/K_i values). For DGIdb, generates REST API queries.
Responder: Synthesizes evidence showing imatinib's primary targets (BCR-ABL, c-KIT, PDGFR) with binding affinities, supported by multiple independent database sources.

Multi-Hop Reasoning Query (`GRAPH` Mode)

Query

```
drugclaw run -query "What are the cardiovascular risks of combining
metformin with an SGLT2 inhibitor in a CYP2C9 poor metabolizer?" -mode
GRAPH
```

Planner: Decomposes into three sub-questions spanning DDI (metformin + SGLT2 inhibitor interaction), ADR (cardiovascular adverse events), and PGx (CYP2C9 metabolizer status). Selects skills from MecDDI, FAERS, PharmGKB, and DrugBank.
Graph Builder: Extracts entities (metformin, empagliflozin/dapagliflozin, CYP2C9) and relationships (metabolized_by, interacts_with, causes) into a query-specific knowledge graph.
Reranker: Evaluates multi-hop paths such as *metformin* $\xrightarrow{\text{interacts_with}}$ *SGLT2 inhibitor* $\xrightarrow{\text{metabolized_by}}$ *CYP2C9* $\xrightarrow{\text{affects}}$ *cardiovascular risk*, scoring by structural and semantic relevance.
Reflector: Evaluates evidence sufficiency across all three sub-domains. If PGx evidence is insufficient ($\sigma_1 < 0.7$), triggers a second retrieval iteration focused on PharmGKB and web search.

Web-Only Fallback (`WEB_ONLY` Mode)

Query

```
drugclaw run -query "Latest clinical trial results for semaglutide in
NASH?" -mode WEB_ONLY
```

Web Search Agent: Dispatches parallel searches to PubMed (for peer-reviewed results), ClinicalTrials.gov (for ongoing/completed trials), and DuckDuckGo (for recent news). Deduplicates results by URL, generates LLM summaries, and produces a synthesized answer with timestamped citations.

B Design Decisions and Comparison

A central design decision in DrugClaw is the commitment to resource-native querying over uniform abstraction. This choice introduces additional complexity, as each skill requires its own documentation, examples, and fallback scripts, but yields significant benefits in three respects. First, *expressiveness*: native API access preserves the full query capabilities of each database (e.g., GraphQL for Open Targets, SPARQL for RDF stores, specialized filtering for ChEMBL bioactivity data). Second, *provenance*: source-specific metadata is preserved, enabling precise citation and traceability. Third, *extensibility*: new resources can be added by

creating a skill directory without modifying the core framework. The trade-off is increased maintenance burden: each skill requires a `SKILL.md`, `example.py`, `retrieve.py`, and the skill implementation file. This is mitigated through the standardized skill structure and the three-tier fallback mechanism.

The vibe coding approach, having the Code Agent read documentation and generate code dynamically, is inherently less predictable than fixed tool interfaces. This is addressed through static validation (AST analysis, length limits, and forbidden construct detection), sandboxed execution (restricted builtins, limited imports, and output caps), and the three-tier fallback (deterministic CLI scripts and direct method calls as safety nets).

The conditional graph reasoning mechanism (Section 3.4) balances accuracy and efficiency. The `_should_run_graph()` decision function considers entity count (multi-entity queries benefit from graph-based relationship discovery), question type (mechanistic and interaction queries benefit from multi-hop paths), and evidence volume (sparse evidence may not support meaningful graph construction). The reflection mechanism (Section 3.5) uses conservative stopping conditions ($\sigma_{\min} = 0.7$, $\delta_{\min} = 0.1$, $T_{\max} = 2$) to balance thoroughness and latency. The maximum iteration limit of 2 reflects the empirical observation that most queries achieve sufficient evidence within two retrieval cycles, with diminishing returns thereafter.

Table 3 provides a qualitative comparison of DrugClaw with related systems along key architectural dimensions. This comparison is based on published system descriptions rather than empirical benchmarking.

Table 3: Qualitative comparison of DrugClaw with related frameworks along key architectural dimensions.

Feature	**DrugClaw**	**ChemCrow**	**Vanilla RAG**	**DrugAgent**	**BiomedRAG**
Domain focus	Drug-native	Chemistry	Generic	Drug disc.	Biomedical
# Data sources	57 skills	18 tools	1–3	Variable	1–3
Source-native access	✓	Partial	×	×	×
Multi-agent	8 agents	1 agent	×	Multi	×
Graph reasoning	✓	×	×	×	×
Iterative reflection	✓	×	×	×	×
Evidence traceability	Full attribution	Partial	Passage-level	Limited	Passage
Fallback mechanism	3-tier	×	×	×	×
Code generation	Vibe coding	×	×	×	×

C Limitations and Future Work

DrugClaw relies on external APIs for many skills, introducing latency and rate-limiting constraints. Future work will explore caching strategies, local database mirrors, and the existing Hugging Face mirror (`Mike2481/DrugClaw_resources_data`) for offline operation.

The vibe coding approach, while expressive, may occasionally produce incorrect API calls. The three-tier fallback mechanism mitigates but does not eliminate this risk. Fine-tuned code generation models or few-shot calibration could improve reliability.

Comprehensive quantitative evaluation of drug intelligence systems is challenging due to the breadth of pharmaceutical domains and the absence of unified benchmarks. Developing domain-specific evaluation suites across all 15 subcategories is an important direction.

DrugClaw does not currently integrate real-time clinical data streams (e.g., electronic health records, real-time adverse event reporting). Integration with clinical data platforms would extend applicability to personalized drug intelligence.

The current reflection mechanism uses LLM-based evaluation for sufficiency scoring. Incorporating reinforcement learning, in the spirit of recent work on RL for agentic systems [24], to optimize skill selection, code generation, and iteration control is a promising future direction. Specifically, the reward signal r_t (Eq. 20) could serve as the basis for policy gradient optimization of the Planner and Retriever agents.

Extending DrugClaw to handle multi-modal evidence, including molecular structures, protein 3D conformations, and clinical imaging data, would broaden the scope of addressable pharmaceutical queries.

D Complete Agent State Specification

The `AgentState` dataclass serves as the shared communication backbone across all agents throughout the LangGraph execution. Table 4 enumerates the full set of fields carried in this state object. These fields collectively capture the user's input, extracted entities, omics constraints, the evolving reasoning history, real-time scoring metrics, accumulated agent outputs, and control flow flags that govern pipeline routing and termination decisions.

Table 4: Complete field specification of the `AgentState` dataclass. Each field is propagated through all LangGraph nodes and updated incrementally as agents execute.

Field	Type	Description
`original_query`	`str`	The user's natural language query
`constraints`	`OmicsConstraints`	Gene sets, pathway sets, disease terms, tissue types
`reasoning_history`	`List[ReasoningStep]`	Ordered reasoning steps with subgraphs and metrics
`current_sufficiency`	`float`	Evidence sufficiency score $\sigma_t \in [0, 1]$
`current_reward`	`float`	Marginal reward r_t
`iteration_count`	`int`	Current iteration number
`agent_outputs`	`Dict[AgentType, Any]`	Accumulated outputs from each agent
`thinking_mode`	`ThinkingMode`	GRAPH, SIMPLE, or WEB_ONLY
`query_plan`	`QueryPlan`	Structured plan from Planner Agent
`evidence_items`	`List[EvidenceItem]`	Normalized evidence items
`subgraph`	`EvidenceSubgraph`	Current knowledge graph
`final_answer`	`FinalAnswer`	Synthesized answer with claims and citations
`control_flags`	`Dict`	Flow control (halt, web_search_needed, etc.)

E LangGraph Execution Paths

In `GRAPH` mode, the execution follows:

$$\begin{aligned}&\texttt{plan} \rightarrow \texttt{retrieve} \rightarrow \texttt{normalize_evidence} \rightarrow \texttt{optional_graph}\\ &\quad \rightarrow \texttt{graph_build} \rightarrow \texttt{rerank} \rightarrow \texttt{assess_claims}\\ &\quad \rightarrow \texttt{respond} \rightarrow \texttt{reflect} \rightarrow [\texttt{web_search}] \rightarrow \texttt{finalize}\end{aligned}$$

In `SIMPLE` mode, the execution follows:

$$\begin{aligned}&\texttt{plan} \rightarrow \texttt{retrieve} \rightarrow \texttt{normalize_evidence}\\ &\quad \rightarrow \texttt{assess_claims} \rightarrow \texttt{simple_respond} \rightarrow \texttt{finalize}\end{aligned}$$

In `WEB_ONLY` mode, the execution follows:

$$\texttt{web_search_direct} \rightarrow \texttt{finalize}$$

Four conditional routing nodes govern the execution flow. The `_route_by_mode` node serves as the entry router, selecting the pipeline based on `thinking_mode`. The `_after_normalize_evidence` node routes to graph construction in `GRAPH` mode or to assessment in `SIMPLE` mode. The `_after_optional_graph` node evaluates whether graph reasoning is justified based on entity count, question type, and evidence volume. The `_after_reflect` node checks the halting condition (Eq. 14) and either routes to finalization or triggers another retrieval iteration.

F Sandbox Security Model

The Code Agent's execution sandbox implements a defense-in-depth strategy that combines static analysis, runtime restrictions, and cascading fallback mechanisms to prevent unsafe code execution. Table 5 summarizes the security layers, spanning compile-time AST validation, runtime builtin and import restrictions, and two tiers of deterministic fallback that activate when dynamically generated code fails validation or execution.

Table 5: Defense-in-depth security model of the Code Agent sandbox. Static layers are enforced before execution; runtime layers constrain the execution environment; fallback layers provide deterministic recovery when code generation fails.

Layer	Mechanism	Constraint
Static	AST analysis	Max 200 nodes, no `while`/`class`/`lambda`
Static	Length check	Max 4,000 characters
Static	Import whitelist	Only `json`, `math`, `re`, `statistics`
Runtime	Builtin restriction	21 safe builtins only
Runtime	Output cap	Max 12,000 characters
Runtime	No system access	`os`, `sys`, `subprocess` blocked
Fallback	Tier 2	Deterministic `retrieve.py`
Fallback	Tier 3	Direct `skill.retrieve()`

G Evidence Item Schema

Each piece of retrieved evidence is normalized into an `EvidenceItem` dataclass before entering downstream processing. Table 6 details the complete field schema. The schema captures provenance information (source skill, type, title, and locator), the extracted textual and structured content (snippet and payload), derived claim-level annotations (claim statement, evidence kind, support direction), and quality metrics (confidence and retrieval scores). This unified representation ensures that heterogeneous outputs from different skills can be compared, aggregated, and assessed on a common basis throughout the claim assessment and response synthesis stages.

Table 6: Complete field schema of the `EvidenceItem` dataclass. All evidence retrieved from heterogeneous skills is normalized into this format before claim assessment and response synthesis.

Field	Type	Description
`evidence_id`	`str`	Unique identifier
`source_skill`	`str`	Originating skill name
`source_type`	`str`	Database, literature, prediction, etc.
`source_title`	`str`	Record title
`source_locator`	`str`	URL or file path
`snippet`	`str`	Relevant text excerpt
`structured_payload`	`Dict`	Skill-specific structured data
`claim`	`str`	Derived claim statement
`evidence_kind`	`str`	experimental, observational, predicted
`support_direction`	`int`	$+1$ (supporting), -1 (contradicting), 0 (neutral)
`confidence`	`float`	Quality score $\in [0, 1]$
`retrieval_score`	`float`	Retrieval relevance score
`timestamp`	`datetime`	Retrieval timestamp
`metadata`	`Dict`	Additional metadata

H System Configuration, Deployment, and Output Formatting

DrugClaw uses a JSON-based configuration file (`navigator_api_keys.json`) supporting environment variable override (`DRUGCLAW_KEY_FILE`). The configuration specifies the LLM backend (API key, base URL, model name, compatible with OpenAI, Azure OpenAI, vLLM, Ollama, Together AI, and any OpenAI-compatible endpoint), default parameters (temperature 0.7, max tokens 2,000, max iterations 2, minimum evidence score 0.7), agent weights (semantic weight 0.6 and structural weight 0.4 for path reranking), and skill configurations (80+ data source integrations with individual timeout values of 15 to 25 seconds and local file paths).

The LLM client wraps the OpenAI API client with DrugClaw-specific capabilities. It provides standard text generation via `chat.completions.create()` with configurable temperature and token limits, JSON generation that appends a JSON-only instruction and uses temperature 0.3 for consistency with multi-stage JSON parsing (direct parsing, markdown fence stripping, regex extraction, and truncation repair), and a truncation repair module that counts unmatched brackets and braces, removes trailing incomplete key-value pairs, and appends closing characters.

DrugClaw provides a comprehensive CLI for environment validation (`drugclaw doctor`), resource and command listing (`drugclaw list`), preset demonstrations covering adverse reactions, drug-target interactions, and drug labeling (`drugclaw demo`), and custom queries with full control over reasoning mode, evidence display, and output format (`drugclaw run`):

```
# Environment validation
drugclaw doctor

# Preset demonstrations
drugclaw demo --preset adr    # Adverse reactions
drugclaw demo --preset dti    # Drug-target interactions

# Custom queries with full control
drugclaw run --query "What are the known targets of
  imatinib and their downstream pathways?" \
  --mode GRAPH \
  --show-evidence --show-plan --show-claims \
  --debug-agents --save-md-report output.md
```

The Python API enables programmatic access:

```
from drugclaw.config import Config
from drugclaw.main_system import DrugClawSystem
from drugclaw.models import ThinkingMode

config = Config(key_file="navigator_api_keys.json")
system = DrugClawSystem(config)

result = system.query(
    "What prescribing info exists for metformin?",
    thinking_mode=ThinkingMode.SIMPLE,
    resource_filter=["DailyMed", "openFDA Human Drug"]
)
# result.answer_text, result.key_claims,
# result.evidence_items, result.citations
```

DrugClaw maintains structured query logs for reproducibility and debugging:

```
query_logs/
  query_index.json
  query_YYYYMMDD_HHMMSS_XXXXXX/
    answer.md          # Final answer
    report.md          # Full execution report
    metadata.json      # Query metadata
    reasoning_trace.md # Step-by-step reasoning
    evidence.json      # Structured evidence
    full_result.pkl    # Complete serialized result
```

The `QueryLogger` provides search, statistics, and CSV export capabilities. A `QuerySession` class manages related queries within a session, tracking duration and query count. The `drugclaw doctor`

command performs comprehensive environment validation, checking configuration file validity and LLM connectivity, required Python imports and dependency availability, resource registry status and health metrics, demo preset availability, and CLI tool installation status.

DrugClaw produces structured output with multiple presentation layers. Evidence confidence is displayed through visual indicators: HIGH ($\gamma \geq 0.7$), MEDIUM ($0.4 \leq \gamma < 0.7$), and LOW ($\gamma < 0.4$), accompanied by ASCII progress bars and numerical precision. Retrieved evidence is formatted in tabular form with columns for source, entities, relationships, confidence, and references, where duplicate evidence rows are merged by case-insensitive keys with occurrence counts. The full reasoning trace, including reward scores, sufficiency metrics, and intermediate answer previews (truncated to 200 characters), is available in collapsible HTML detail elements. Citations are deduplicated by URL and presented as numbered lists, and the `wrap_answer_card()` function assembles the complete output consisting of a metadata header, answer body, evidence table, reasoning trace, source citations, and attribution.